Universidad Politécnica de Chiapas
Ingeniería en Tecnologías de la Información e Innovación Digital (IT2iD)

Reporte técnico de métricas

Medición de métricas en bases documentales: contención de CPU y memoria en agregaciones JSON

MongoDB 9.0.4 (WiredTiger) frente a PostgreSQL 18.6 (JSONB)
bajo límites de Docker cgroups v2

Materia: Bases de Datos en la Nube
Docente: Horacio Irán Solís Cisneros
Alumno(a): Luvia Magali Hidalgo Garcia
Matrícula: 243732
Fecha: 1 de noviembre de 2026

Suchiapa, Chiapas, México

Resumen ejecutivo

Se midió la latencia (P50/P95/P99), el throughput, el tiempo de *throttling* CFS, la memoria y el volcado a disco de dos motores que almacenan documentos JSON —MongoDB 9.0.4 y PostgreSQL 18.6 con JSONB— al ejecutar las dos agregaciones definidas en el playbook (Q1: *unwind*, filtro y agrupación; Q2: ordenamiento Top-1000) sobre un dataset determinista de 500 000 órdenes, bajo tres perfiles de contención (4/2/1 CPU y 4096/2048/1024 MB sin swap) y tres niveles de concurrencia (1, 10 y 50 clientes).

La campaña ejecutó **180 corridas válidas**: la matriz obligatoria de 36 celdas × 3 repeticiones (108 corridas) más 72 corridas de contraste con la variante de memoria ampliada (`work_mem = 64MB` / `allowDiskUse: true`). Cada corrida recreó el contenedor, vació cachés, y midió 300 s tras 60 s de calentamiento. Resultado global: 144 corridas `ok`, 24 con volcado a disco (`spill`) y 12 terminadas por el OOM-killer (`oom`); las 24 y las 12 corresponden *exclusivamente* a PostgreSQL en Q1.

Hallazgos principales:

- **Q1 (agregación sobre arreglos anidados):** MongoDB resolvió los 96 grupos en memoria con 64 KB de estado (`peakTrackedMemBytes`) y sin volcado en ninguna de las 36 corridas. PostgreSQL volcó a disco en todas las corridas que terminaron (entre 30 y 81 GiB de archivos temporales por ventana de 300 s) y **fue terminado por el OOM-killer con 50 clientes en los perfiles B y C**. A $c = 1$, PostgreSQL consumió alrededor de 3 veces más CPU por cada 1 000 documentos que MongoDB (2.9–3.5 veces).
- **Q2 (Top-1000):** ambos motores aplicaron un ordenamiento acotado (≈276 KB en PostgreSQL, ≈1.35 MB en MongoDB) y ninguno volcó a disco con ninguna variante: Q2 funciona como control negativo de memoria. Aquí PostgreSQL fue *más rápido* (Parallel Seq Scan) y MongoDB se degradó fuertemente al reducirse su caché (IXSCAN + 226 680 `FETCH`).
- **Variante de memoria ampliada:** en el perfil C, subir `work_mem` a 64 MB mejoró PostgreSQL con 1 cliente (+24 % QPS) pero convirtió el `spill` de 10 clientes en `oom`, confirmando que `work_mem` es un límite por operación y por proceso, no un presupuesto global.

Desviación declarada respecto al entorno solicitado

El playbook solicita ejecutar sobre Linux. Los motores, los límites de recursos (cgroups v2, CFS, OOM-killer) y la telemetría se ejecutaron sobre un **kernel Linux 6.6** con **Docker Engine 29.1.3**, pero dentro de la máquina virtual de WSL2 que provee Docker Desktop sobre un anfitrión Windows, y el generador de carga (runner) se ejecutó en Windows. La sección 1 detalla el entorno y la sección 6.6 analiza su efecto sobre la validez de los resultados.

Índice general

Resumen ejecutivo **1**

1 Especificaciones del host y del entorno Docker **4**
1.1 Hardware y sistema anfitrión . . . 4
1.2 Imágenes fijadas por digest . . . 4
1.3 Perfiles de recursos (cgroups v2) . . . 4
1.4 Stack de telemetría . . . 5

2 Método de medición y verificación del baseline **5**
2.1 Dataset sintético . . . 5
2.2 Esquemas e índices . . . 5
2.3 Cargas de trabajo . . . 6
2.4 Matriz experimental . . . 6
2.5 Protocolo por corrida . . . 7
2.6 Definición de las métricas . . . 7
2.7 Control de calidad y consolidación . . . 7

3 Resultados consolidados **8**
3.1 Contraste de las variantes de memoria . . . 9

4 Análisis gráfico **10**
4.1 P99 frente a concurrencia . . . 10
4.2 Throttling CFS frente a throughput . . . 10
4.3 Evolución de la memoria . . . 11

5 Planes de ejecución **11**
5.1 Q1 en PostgreSQL: estimación errónea, ordenamiento completo y volcado . . . 12
5.2 Q1 en MongoDB: agregación en flujo . . . 12
5.3 Q2 en ambos motores: Top-N acotado y patrón de acceso . . . 12

6 Interpretación: límites de memoria, volcado a disco y throttling de cgroups **13**
6.1 M1 — ¿Cómo cambian P95/P99 al limitar CPU y memoria? . . . 13
6.2 M2 — ¿Qué motor gestiona mejor los buffers antes del volcado o del OOM? . . . 14
6.3 M3 — Huella de CPU y memoria por cada 1 000 documentos . . . 15
6.4 Throttling de cgroups y paralelismo intra-consulta . . . 16
6.5 Respuestas a las preguntas del revisor . . . 16
6.6 Amenazas a la validez y limitaciones . . . 16

7 Recomendaciones técnicas para despliegues documentales en la nube **17**
7.1 Respaldadas directamente por los resultados . . . 17
7.2 Buenas prácticas generales (no medidas directamente) . . . 18

A Registro de incidencias de la campaña **18**

B Reproducibilidad **18**

C Esquema del CSV de mediciones **19**

Índice de tablas

1 Hardware y software del entorno de ejecución. 4
2 Imágenes de contenedor utilizadas. 4
3 Perfiles de contención aplicados. 4
4 Características verificadas del dataset. 5
5 Composición de la campaña de 180 corridas. 6
6 Resultados de Q1 (variante `strict`): mediana de 3 repeticiones. 8
7 Resultados de Q2 (variante `strict`): mediana de 3 repeticiones. 8
8 Variante `strict` frente a `relaxed` (anexo, 72 corridas). Mediana de 3 repeticiones. . 9
9 M1: P99 (s) por perfil y variación respecto del perfil A, a igual concurrencia (variante `strict`). 13
10 M2: estado de las 3 repeticiones por celda y volumen de archivos temporales de PostgreSQL en la ventana (mediana). 14
11 M3 (variante `strict`, mediana de 3 repeticiones). Mo = MongoDB, PG = PostgreSQL. 15
12 Eventos operativos registrados en el journal. 18

Índice de figuras

1 Latencia P99 frente a concurrencia (escalas logarítmicas). Mediana de 3 repeticiones; las barras indican el rango [mín, máx]. Las cruces señalan las celdas terminadas por OOM, que no tienen P99. 10
2 Tiempo de *throttling* CFS en la ventana de 300 s frente al throughput, por corrida (variante `strict`). 10
3 Memoria del motor (`anon + shmem`, línea continua) y caché de páginas (`file`, punteada) en Q1 con 50 clientes, repetición 1. La línea roja es el límite del cgroup; la franja gris, la ventana de medición. 11
4 Throughput con la variante `strict` (sólido) y `relaxed` (rayado) en el perfil C. . . . 11
5 M3 con un cliente: CPU y memoria integrada por cada 1 000 documentos. 15

1 Especificaciones del host y del entorno Docker

1.1 Hardware y sistema anfitrión

Tabla 1: Hardware y software del entorno de ejecución.

Procesador	Intel Core i7-1355U (13.ª gen.), arquitectura híbrida: 10 núcleos (2 P-cores + 8 E-cores), 12 hilos. Equipo portátil.
Sistema anfitrión	Windows (x64) con Windows Subsystem for Linux 2 (WSL2, hipervisor Hyper-V).
Kernel donde corren los contenedores	`Linux 6.6.87.2-microsoft-standard-WSL2` (VM de Docker Desktop).
Memoria de la VM Linux	7 984 328 kB (≈7.6 GiB) `MemTotal`.
Runtime de contenedores	Docker Desktop 4.55.0 con Docker Engine 29.1.3 (API 1.52), containerd 2.2.0, runc 1.3.4, Docker Compose 2.40.3.
Control de recursos	cgroups v2 (jerarquía unificada), driver `cgroupfs`, almacenamiento `overlayfs`.
Runner (generador de carga)	Python multiproceso (`psycopg` 3 y `pymongo`), un proceso por cliente, ejecutado en Windows.

1.2 Imágenes fijadas por digest

Para que la medición sea reproducible no se usó ninguna etiqueta mutable (`latest`); todas las imágenes se fijaron por digest SHA-256 en `docker-compose.yml` y la versión reportada por cada motor se guardó en el `meta.json` de cada corrida.

Tabla 2: Imágenes de contenedor utilizadas.

Servicio	Imagen	Digest (prefijo)
MongoDB	`mongo:9.0.4`	`sha256:452bdaf2127b...`
PostgreSQL	`postgres:18.6` (18.6-1.pgdg13+2)	`sha256:979bccfde354...`
cAdvisor	`ghcr.io/google/cadvisor:v0.60.6`	`sha256:b8e7d1093144...`
Prometheus	`prom/prometheus:v3.15.1`	`sha256:fa3302279710...`
Node Exporter	`prom/node-exporter:v1.12.1`	`sha256:1b4e4438faca...`

1.3 Perfiles de recursos (cgroups v2)

Los tres perfiles del playbook se aplicaron con `cpus` (cuota CFS en `cpu.max`), `mem_limit` (`memory.max`) y `memswap_limit = mem_limit`, que fija `memory.swap.max = 0`. En cgroups v2 el parámetro `mem_swappiness` que menciona el playbook no tiene efecto; se mantuvo en 0 por trazabilidad, pero la ausencia de swap la garantiza `memswap_limit`. Los contenedores de los motores se fijaron al `cpuset 0-3` y la telemetría a `4,5,10,11`.

Las cachés internas se dimensionaron con la regla por defecto de cada fabricante aplicada al límite del contenedor: `shared_buffers` = 25 % de la RAM (PostgreSQL) y `wiredTigerCacheSizeGB` = $\max(0.25,\ 0.5 \times (\text{RAM} - 1\,\text{GB}))$ (MongoDB). La variable independiente del experimento es el límite del cgroup; dentro de él, cada motor se configuró como se desplegaría en producción.

Tabla 3: Perfiles de contención aplicados.

ID	Nombre	CPUs	`cpu.max`	Memoria	Swap	`shared_buffers`	Caché WT
`PROFILE_A_BASELINE`	Sin contención	4.0	400000 100000	4096 MB	0	1024 MB	1.5 GB
`PROFILE_B_MEDIUM`	Contención media	2.0	200000 100000	2048 MB	0	512 MB	0.5 GB
`PROFILE_C_SEVERE`	Contención severa	1.0	100000 100000	1024 MB	0	256 MB	0.25 GB

Los límites efectivos se verificaron en cada corrida con `docker inspect` (por ejemplo, perfil A: `NanoCpus = 4 000 000 000`, `Memory = MemorySwap = 4 294 967 296`, `CpusetCpus = 0-3`) y

quedaron registrados en `meta.json`.

1.4 Stack de telemetría

- **cAdvisor** (métricas de contenedor de cgroups v2) con `--housekeeping_interval=500ms`, `--max_housekeeping_interval=1s` y `--allow_dynamic_housekeeping=false`.
- **Prometheus** con `scrape_interval: 1s` y `honor_timestamps: false` (sin esta opción, el *jitter* de cAdvisor producía sólo ≈197 muestras en 300 s durante el piloto).
- **Node Exporter** para métricas de la VM Linux (E/S, disco, swap).
- **Muestreador directo de cgroups v2** a 1 Hz (`cpu.stat`, `memory.stat`, `memory.events`, `memory.peak`, `io.stat`) como segunda fuente: conserva el estado aunque el contenedor muera por OOM antes del siguiente *scrape*.

2 Método de medición y verificación del baseline

2.1 Dataset sintético

El generador `synthetic_data_generator.py` produce un flujo NDJSON determinista (semilla 42) que cumple la sección 4 del playbook: documentos de órdenes con tres niveles de anidamiento y un promedio de tres ítems.

Tabla 4: Características verificadas del dataset.

Documentos	500 000
Tamaño NDJSON	557 058 284 bytes (1 114.1 bytes/doc)
SHA-256 del flujo	`9cd202a6e102ad8e...c2c0e6f`
Ítems totales (desanidados)	1 499 046 (2.998 por orden)
Ítems con `price > 50` (Q1)	644 159 (42.97 %)
Grupos de Q1 (8 países × 12 categorías)	96
Órdenes `completed`	399 714 (79.94 %)
Órdenes con `timestamp` ≥ 2026-01-01	226 680 (45.34 %)
Órdenes que cumplen el filtro de Q2	181 307 (36.26 %)

Atributos: `order_id` (UUID), `timestamp` (ISO-8601 UTC), `customer` (`id`, `tier`, `segment`, `geo{country, state, city, zip, coordinates}`), `items[]` (`sku`, `category`, `brand`, `qty`, `price`, `tax_pct`, `discounts`), `payment` (`method`, `status`, `transaction_id`, `gateway_latency_ms`, `totals`, `final_total`) y `metadata` (`channel`, `user_agent`, `session_id`, `flags`). Se incluyó `payment.final_total` porque Q2 lo usa, aunque la sección 4 del playbook sólo lista `totals`.

Verificación del baseline (mismo dataset en ambos motores). Tras la carga, ambos motores devolvieron los mismos totales: `count = 500 000`, `completed = 399 714` y $\sum$`final_total` = 229 397 032.51. En PostgreSQL se ejecutó `VACUUM (FREEZE, ANALYZE)` tras el `COPY` para que ni *autovacuum* ni los *hint bits* escribieran durante las mediciones.

2.2 Esquemas e índices

- **MongoDB:** colección `orders` sin sharding (WiredTiger), índices simples `timestamp_1` y `order_id_1`.
- **PostgreSQL:** tabla `orders (id UUID PRIMARY KEY, data JSONB)` e índice B-Tree sobre `((data->>'timestamp')::timestamptz)`.

El cast `text::timestamptz` es `STABLE` (depende del parámetro `TimeZone`) y PostgreSQL no permite indexarlo. Para respetar sin cambios el SQL de Q2 de la sección 5.2, en el contenedor

(fijado en `TZ=UTC`) se marcó `timestamptz_in` como `IMMUTABLE`. Esta modificación del catálogo no es posible en PostgreSQL administrado (RDS, Cloud SQL); se discute en la sección 7. Como muestra la sección 5, el planificador no usó este índice en Q2.

2.3 Cargas de trabajo

Las consultas se ejecutaron exactamente como las define el playbook (sección 5).

Q1 — Unwind, Filter & Grouping (saturación de CPU).

```
[ { "$unwind": "$items" },
  { "$match": { "items.price": { "$gt": 50 } } },
  { "$group": { "_id": { "country": "$customer.geo.country", "category": "$items.category" },
      "total_revenue": { "$sum": { "$multiply": ["$items.qty", "$items.price"] } },
      "avg_revenue":   { "$avg": { "$multiply": ["$items.qty", "$items.price"] } },
      "count": { "$sum": 1 } } } ]
```

```
SELECT data->'customer'->'geo'->>'country' AS country, item->>'category' AS category,
       SUM((item->>'qty')::numeric * (item->>'price')::numeric) AS total_revenue,
       AVG((item->>'qty')::numeric * (item->>'price')::numeric) AS avg_revenue,
       COUNT(*) AS count
FROM orders, jsonb_array_elements(data->'items') AS item
WHERE (item->>'price')::numeric > 50
GROUP BY 1, 2;
```

Q2 — Massive Sort & Top-K (estrés de buffer).

```
[ { "$match": { "payment.status": "completed", "timestamp": { "$gte": "2026-01-01T00:00:00Z" } } },
  { "$sort": { "payment.final_total": -1 } }, { "$limit": 1000 },
  { "$project": { "order_id": 1, "customer.id": 1, "payment.final_total": 1 } } ]
```

```
SELECT data->>'order_id' AS order_id, data->'customer'->>'id' AS customer_id,
       (data->'payment'->>'final_total')::numeric AS final_total
FROM orders
WHERE data->'payment'->>'status' = 'completed'
  AND (data->>'timestamp')::timestamptz >= '2026-01-01 00:00:00Z'
ORDER BY (data->'payment'->>'final_total')::numeric DESC
LIMIT 1000;
```

Variantes de memoria. El playbook define dos variantes para el estrés de memoria. Se aplicaron como parámetro de sesión (PostgreSQL, una sola vez al abrir la conexión) o de la operación (MongoDB):

- `strict`: `SET work_mem = '4MB'` / `allowDiskUse: false`.
- `relaxed`: `SET work_mem = '64MB'` / `allowDiskUse: true`.

2.4 Matriz experimental

Tabla 5: Composición de la campaña de 180 corridas.

Bloque	Factores	Celdas	Corridas
Matriz principal (`strict`)	2 motores × 3 perfiles × 2 consultas × 3 concurrencias	36	108
Anexo `relaxed`, perfil C	2 motores × {Q1, Q2} × 3 concurrencias	12	36
Anexo `relaxed`, Q2 en A y B	2 motores × {A, B} × 3 concurrencias	12	36
Total		60	180

Con esto Q2 queda medida con ambas variantes en los tres perfiles y las tres concurrencias, como pide la sección 5.2; Q1 `relaxed` en el perfil C se añadió porque es la consulta donde hay volcado y OOM (pregunta M2). Las filas `relaxed` se registraron en `anexo_variantes.csv` con `query_id = Q1_relaxed / Q2_relaxed`, sin alterar el esquema de 14 columnas. El orden de las corridas se aleatorizó por bloques de repetición con semilla fija (20261030).

2.5 Protocolo por corrida

1. Detener y **recrear** el contenedor del motor con los límites del perfil (vacía `shared_buffers` y la caché de WiredTiger, y crea un cgroup nuevo).
2. Vaciar la caché de páginas del kernel: `sync; echo 3 > /proc/sys/vm/drop_caches`.
3. Lanzar c clientes, cada uno en un proceso, que ejecutan la consulta en lazo cerrado (sin pausas) con una conexión propia.
4. **Warmup** de 60 s (no se mide), **medición** de 300 s y **cooldown** de 60 s.

Cada corrida produjo una fila del CSV y un `meta.json` con el tamaño de muestra, los límites reales, la versión del motor, los contadores internos y las métricas M3. La duración típica fue de ≈7.1 minutos por corrida.

2.6 Definición de las métricas

- **Throughput**: consultas completadas cuyo fin cae dentro de la ventana, dividido entre 300 s.
- **Latencias P50/P95/P99**: percentiles (interpolación lineal) de las latencias de esas consultas. Son percentiles reales, no promedios.
- `cpu_throttled_seconds`: diferencia de `container_cpu_cfs_throttled_seconds_total` entre el inicio y el fin de la ventana, contrastada con `throttled_usec` de `cpu.stat` (ambas fuentes debían coincidir). El kernel acumula este tiempo *por cada CPU* del `cpuset`, por lo que puede superar los 300 s de la ventana (sección 6.4).
- `max_rss_mb`: máximo de `container_memory_rss` (memoria anónima) en la ventana.
- `disk_spill`: verdadero si alguna de tres señales lo indica en la ventana: (1) contadores internos (`temp_files`/`temp_bytes` en `pg_stat_database`; `metrics.query.sort/group` en MongoDB); (2) logs del motor (`temporary file:` con `log_temp_files=0`; `"usedDisk":true`); (3) escrituras de bloque del cgroup ≥32 MiB en `io.stat` con una carga de sólo lectura. La tercera señal cubre el caso en que un OOM reinicia `pg_stat_database` a cero.
- `status`: `oom` si `memory.events` registró `oom_kill` (como el contenedor se recrea en cada corrida, el contador empieza en cero y no hay falsos positivos por el indicador `State.OOMKilled` de Docker); `spill` si hubo volcado; `ok` en otro caso. Así se representa el KPI `oom_killed` del playbook dentro del esquema CSV fijo.
- **M3**: $\mathrm{CPU}_{1k} = \Delta\text{cpu_usage_s}/(\text{consultas} \times 500)$; $\mathrm{Mem}_{\text{peak}} = \max_t(\text{anon} + \text{shmem})$; $\mathrm{Mem}_{1k} = \int(\text{anon} + \text{shmem})\,dt/(\text{consultas} \times 500)$. Se usa `anon + shmem` porque WiredTiger vive en memoria anónima y `shared_buffers` en memoria compartida; se excluye la caché de páginas (`file`).

2.7 Control de calidad y consolidación

Una corrida sólo se registró si tuvo al menos 240 de 300 muestras tanto en Prometheus como en el muestreador de cgroup, y si ambas fuentes de *throttling* coincidían. Las corridas que no cumplieron se descartaron y se repitieron automáticamente (anexo A). En las 180 corridas registradas:

- Prometheus tuvo entre 286 y 301 muestras y el muestreador de cgroup entre 275 y 294 (mediana 291).

- La integral de memoria cubrió entre 295.6 s y 305.3 s de los 300 s.
- Ninguna corrida fue marcada como ruidosa por el control de CPUs ociosas.

Cada celda se resume con la **mediana** de sus 3 repeticiones y el **rango [mín, máx]**. Cuando una repetición completó menos de 100 consultas, su P99 depende de muy pocas observaciones; esas celdas se marcan con † y su P99 debe leerse como indicativo.

3 Resultados consolidados

Las tablas 6 y 7 presentan la matriz principal (variante `strict`, 108 corridas). Las latencias están en segundos; n es el menor número de consultas completadas entre las 3 repeticiones; *Thr.* es el tiempo de *throttling* CFS en la ventana; RSS es `max_rss_mb`.

Tabla 6: Resultados de Q1 (variante `strict`): mediana de 3 repeticiones.

Motor	Perfil	c	QPS	P50 (s)	P95 (s)	P99 (s) [mín–máx]	n	Thr. (s)	RSS (MB)	Estado
MongoDB	A	1	0.647	1.51	1.72	1.95 [1.83–2.04]	190	0.0	681	ok:3
		10	1.380	7.26	8.17	8.72 [8.34–14.32]	301	2.4	738	ok:3
		50	1.333	36.72	41.11	42.50 [36.99–46.65]	399	2.7	922	ok:3
	B	1	0.610	1.58	1.88	2.15 [1.77–2.54]	175	0.0	663	ok:3
		10	0.847	11.82	13.83	14.80 [14.12–16.01]	231	596.8	715	ok:3
		50	0.863	57.10	61.44	63.19 [62.43–68.35]	250	594.6	803	ok:3
	C	1	0.517	1.89	2.27	2.60 [2.47–2.65]	153	11.6	427	ok:3
		10	0.403	24.51	28.85	31.35 [26.62–31.72]	109	894.4	453	ok:3
		50	0.500	112.0	123.3	127.1 [126.0–131.2]	136	892.7	607	ok:3
PostgreSQL	A	1	0.333	2.99	3.28	7.97 [3.46–9.32]†	89	0.0	102	spill:3
		10	0.193	34.03	73.16	83.19 [77.85–217.8]†	41	0.1	568	spill:3
		50	0.180	187.4	188.9	204.6 [194.8–210.0]†	53	0.1	1 980	spill:3
	B	1	0.300	3.31	3.69	3.82 [3.65–17.13]†	62	102.8	102	spill:3
		10	0.193	48.81	56.52	59.29 [53.29–73.03]†	51	446.3	569	spill:3
		50	0.000	—	—	—†	0	424.1	1 752	oom:3
	C	1	0.143	6.89	7.68	8.28 [8.16–15.44]†	43	218.8	100	spill:3
		10	0.090	113.2	146.2	146.8 [110.7–163.4]†	20	739.6	564	spill:3
		50	0.000	—	—	—†	0	786.5	956	oom:3

† Alguna repetición completó menos de 100 consultas: el P99 es indicativo. Las celdas `oom` no completaron ninguna consulta en la ventana, por lo que no tienen percentiles.

Tabla 7: Resultados de Q2 (variante `strict`): mediana de 3 repeticiones.

Motor	Perfil	c	QPS	P50 (s)	P95 (s)	P99 (s) [mín–máx]	n	Thr. (s)	RSS (MB)	Estado
MongoDB	A	1	1.867	0.53	0.60	0.71 [0.65–0.74]	558	0.0	687	ok:3
		10	4.943	2.04	2.64	2.84 [2.65–2.96]	1430	1.6	712	ok:3
		50	4.753	10.43	11.16	11.53 [11.52–11.78]	1405	2.2	803	ok:3
	B	1	0.390	2.51	2.88	3.24 [3.08–3.32]	116	0.0	624	ok:3
		10	0.767	12.89	15.69	16.98 [11.28–21.36]	184	585.1	623	ok:3
		50	0.653	85.53	95.25	98.31 [88.04–99.49]	195	594.4	721	ok:3
	C	1	0.177	5.50	6.42	6.53 [4.94–6.73]†	52	17.0	387	ok:3
		10	0.333	29.61	50.79	55.05 [33.50–73.70]†	69	820.6	387	ok:3
		50	0.167	262.8	320.9	334.4 [298.9–356.3]†	45	883.5	479	ok:3
PostgreSQL	A	1	4.013	0.25	0.28	0.29 [0.26–0.30]	1187	0.0	35	ok:3
		10	4.950	1.72	3.17	3.42 [3.10–3.93]	1473	0.7	175	ok:3
		50	5.467	9.93	10.70	11.55 [10.00–12.20]	1529	1.0	586	ok:3
	B	1	2.693	0.37	0.49	0.57 [0.46–0.84]	689	316.4	35	ok:3
		10	2.217	3.81	7.39	7.66 [7.46–8.08]	644	539.6	175	ok:3
		50	2.787	17.69	23.22	30.74 [27.30–31.97]	741	582.9	581	ok:3
	C	1	1.177	0.81	1.00	1.09 [1.09–1.21]	339	552.2	35	ok:3
		10	0.773	9.89	20.67	21.96 [21.61–22.67]	229	643.0	175	ok:3
		50	0.543	93.37	182.4	196.7 [180.2–273.0]†	96	716.5	575	ok:3

† Alguna repetición completó menos de 100 consultas: el P99 es indicativo.

Lectura de Q1. MongoDB completó todas las celdas (`ok:3` en las 9) con un P99 de 1.95 a 2.60 s a 1 cliente y un throughput prácticamente constante con la concurrencia (el motor ya está saturado de CPU con pocos clientes). PostgreSQL volcó a disco en todas las celdas que terminaron y obtuvo entre 1.9 y 7.4 veces menos throughput que MongoDB; con 50 clientes fue terminado por el OOM-killer en los perfiles B y C en las 6 repeticiones (24 y 244 eventos `oom_kill` acumulados, respectivamente). Con 50 clientes en el perfil A completó, pero con un P99 de 204.6 s: casi toda la ventana.

Lectura de Q2. Ambos motores terminaron las 18 celdas sin volcado. PostgreSQL fue más rápido en casi todas: a 1 cliente en el perfil A obtuvo 4.01 QPS frente a 1.87 de MongoDB. La diferencia se amplía al reducir la memoria: MongoDB pasó de 1.87 a 0.18 QPS a 1 cliente entre los perfiles A y C (−91 %), mientras PostgreSQL pasó de 4.01 a 1.18 (−71 %). La sección 5 explica la causa (patrón de acceso del plan).

3.1 Contraste de las variantes de memoria

Tabla 8: Variante `strict` frente a `relaxed` (anexo, 72 corridas). Mediana de 3 repeticiones.

Q	Motor	Perfil	c	QPS strict	QPS relaxed	ΔQPS	P99 strict (s)	P99 relaxed (s)	Estado strict	Estado relaxed
Q1	MongoDB	C	1	0.517	0.580	+12 %	2.60	1.97	ok:3	ok:3
Q1	MongoDB	C	10	0.403	0.487	+21 %	31.35	25.11	ok:3	ok:3
Q1	MongoDB	C	50	0.500	0.500	+0 %	127.1	112.8	ok:3	ok:3
Q1	PostgreSQL	C	1	0.143	0.177	+24 %	8.28	6.05	spill:3	spill:3
Q1	PostgreSQL	C	10	0.090	0.000	-100 %	146.8	—	spill:3	oom:3
Q1	PostgreSQL	C	50	0.000	0.000	—	—	—	oom:3	oom:3
Q2	MongoDB	A	1	1.867	1.877	+1 %	0.71	0.73	ok:3	ok:3
Q2	MongoDB	A	10	4.943	4.653	-6 %	2.84	2.86	ok:3	ok:3
Q2	MongoDB	A	50	4.753	4.557	-4 %	11.53	12.17	ok:3	ok:3
Q2	MongoDB	B	1	0.390	0.383	-2 %	3.24	3.34	ok:3	ok:3
Q2	MongoDB	B	10	0.767	0.793	+3 %	16.98	18.72	ok:3	ok:3
Q2	MongoDB	B	50	0.653	0.643	-2 %	98.31	98.00	ok:3	ok:3
Q2	MongoDB	C	1	0.177	0.197	+11 %	6.53	5.67	ok:3	ok:3
Q2	MongoDB	C	10	0.333	0.400	+20 %	55.05	26.70	ok:3	ok:3
Q2	MongoDB	C	50	0.167	0.167	+0 %	334.4	302.5	ok:3	ok:3
Q2	PostgreSQL	A	1	4.013	3.930	-2 %	0.29	0.31	ok:3	ok:3
Q2	PostgreSQL	A	10	4.950	4.937	-0 %	3.42	3.40	ok:3	ok:3
Q2	PostgreSQL	A	50	5.467	5.423	-1 %	11.55	11.24	ok:3	ok:3
Q2	PostgreSQL	B	1	2.693	2.377	-12 %	0.57	0.50	ok:3	ok:3
Q2	PostgreSQL	B	10	2.217	2.327	+5 %	7.66	7.60	ok:3	ok:3
Q2	PostgreSQL	B	50	2.787	2.890	+4 %	30.74	28.53	ok:3	ok:3
Q2	PostgreSQL	C	1	1.177	1.297	+10 %	1.09	1.00	ok:3	ok:3
Q2	PostgreSQL	C	10	0.773	0.763	-1 %	21.96	21.44	ok:3	ok:3
Q2	PostgreSQL	C	50	0.543	0.540	-1 %	196.7	186.7	ok:3	ok:3

En las celdas sin volcado (todas las de MongoDB y todas las de Q2) la variante no cambia el plan; las diferencias de hasta ≈ ±20 % reflejan la variabilidad entre corridas del entorno, no un efecto de la variante.

4 Análisis gráfico

4.1 P99 frente a concurrencia

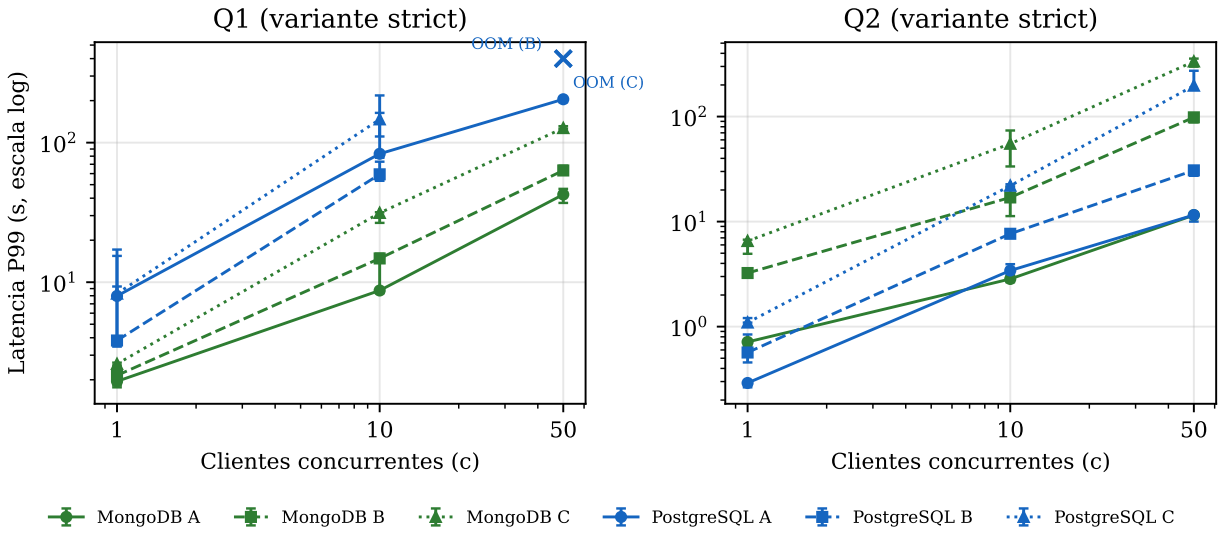


Figura 1: Latencia P99 frente a concurrencia (escalas logarítmicas). Mediana de 3 repeticiones; las barras indican el rango [mín, máx]. Las cruces señalan las celdas terminadas por OOM, que no tienen P99.

En la figura 1 las pendientes son casi paralelas: con un motor saturado de CPU y clientes en lazo cerrado, la latencia crece proporcionalmente al número de clientes (ley de Little: latencia $\approx c/\text{QPS}$). Lo que distingue a los motores es el nivel y el punto de quiebre:

- En Q1, la curva de PostgreSQL está entre 2 y 10 veces por encima de la de MongoDB en cada perfil y desaparece a $c = 50$ en los perfiles B y C (OOM).
- En Q2, MongoDB en el perfil C es la curva más alta (334 s de P99 con 50 clientes), por encima de PostgreSQL en el mismo perfil (197 s).

4.2 Throttling CFS frente a throughput

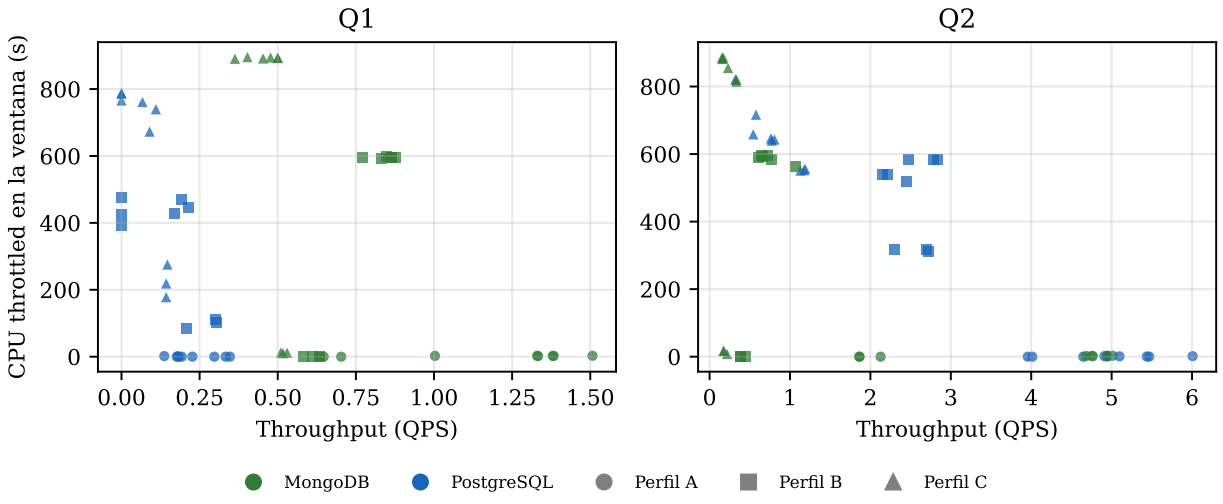


Figura 2: Tiempo de *throttling* CFS en la ventana de 300 s frente al throughput, por corrida (variante `strict`).

La figura 2 muestra tres bandas horizontales: ≈0 s (perfil A), ≈600 s (perfil B) y ≈900 s (perfil C). Esos valores son el techo teórico del *throttling* cuando el contenedor tiene trabajo listo en las 4 CPUs de su `cpuset` durante toda la ventana: $(|\texttt{cpuset}| - \text{cuota}) \times 300\,\text{s}$, es decir $(4-2)\times 300 = 600\,\text{s}$ y $(4-1)\times 300 = 900\,\text{s}$. MongoDB se acerca a ese techo con 10 o más clientes en B y C (585–597 s y 821–894 s); PostgreSQL lo alcanza incluso con un solo cliente en Q2 (316 s en B y 552 s en C), porque cada consulta lanza un líder y 2 *workers* paralelos que compiten por la misma cuota.

4.3 Evolución de la memoria

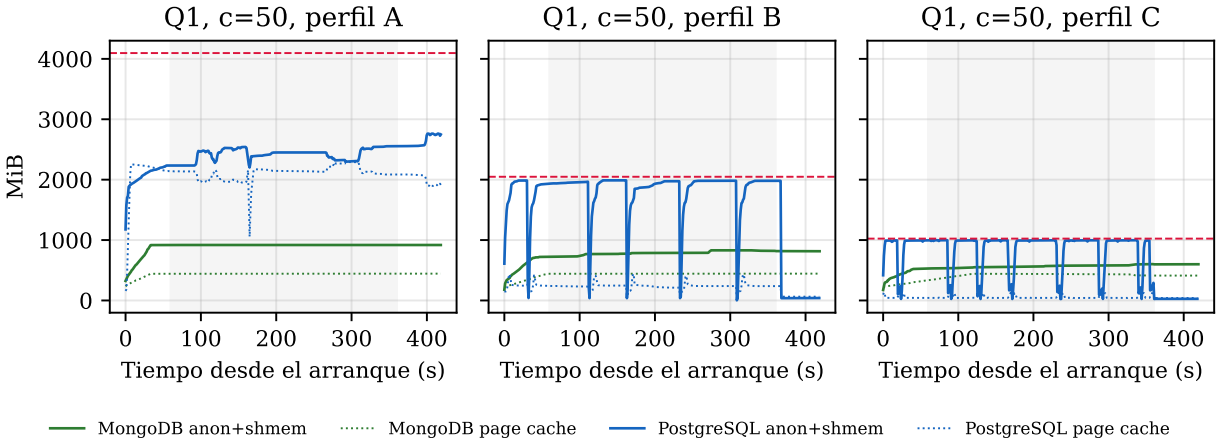


Figura 3: Memoria del motor (`anon + shmem`, línea continua) y caché de páginas (`file`, punteada) en Q1 con 50 clientes, repetición 1. La línea roja es el límite del cgroup; la franja gris, la ventana de medición.

La figura 3 es la evidencia directa de M2. MongoDB se mantiene plano por debajo de 1 GB en los tres perfiles. PostgreSQL crece hasta ≈2.6 GB en el perfil A (dentro de los 4 GB) y en los perfiles B y C dibuja un patrón de diente de sierra: la memoria sube hasta el límite del cgroup, el kernel mata un *backend*, el `postmaster` reinicia todos los procesos (recuperación) y el ciclo se repite durante toda la ventana.

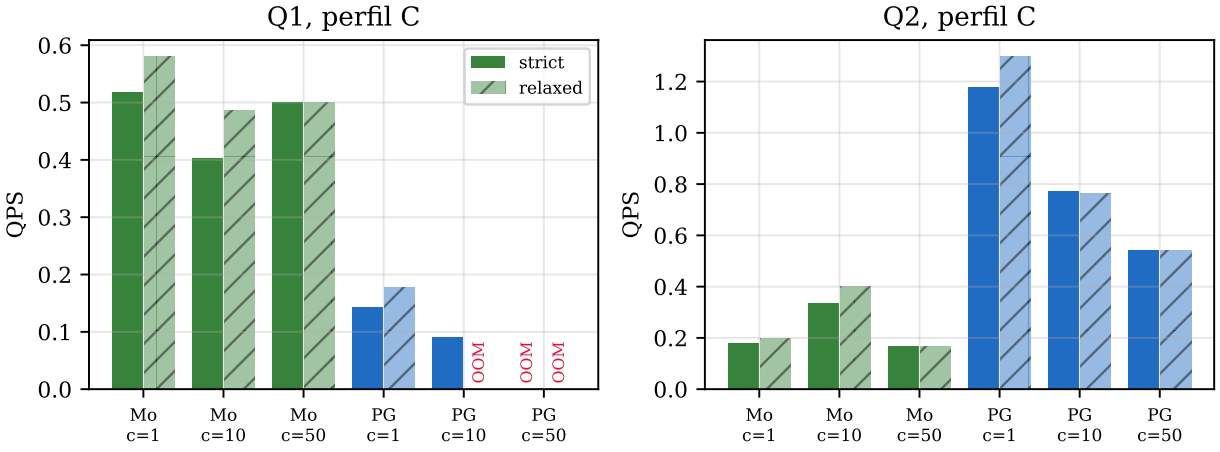


Figura 4: Throughput con la variante `strict` (sólido) y `relaxed` (rayado) en el perfil C.

5 Planes de ejecución

Los planes se obtuvieron con `explain("executionStats")` en MongoDB y `EXPLAIN (ANALYZE, BUFFERS, VERBOSE)` en PostgreSQL, fuera de la ventana de medición, en cada perfil y variante (24 planes en `results/explain/`). Los que se citan son del perfil C salvo que se indique otro.

5.1 Q1 en PostgreSQL: estimación errónea, ordenamiento completo y volcado

```
GroupAggregate (rows=16504191 estimadas; actual rows=96)
  -> Incremental Sort   Pre-sorted Groups: 8  Sort Method: external merge  Peak Disk: 34336kB
     -> Nested Loop (actual rows=644159)
        -> Gather Merge   Workers Launched: 2   (rows=500000, width=1460)
           -> Sort  Sort Method: external merge  Disk: 221680kB   (líder)
                    Worker 0: external merge  Disk: 248680kB
                    Worker 1: external merge  Disk: 251064kB
              -> Parallel Seq Scan on orders (actual rows=166666 x 3)
        -> Function Scan on jsonb_array_elements item
              Filter: ((item.value->>'price')::numeric > 50)  Rows Removed by Filter: 2
Execution Time: 8041 ms   (perfil A: 5654 ms)
```

El planificador no tiene estadísticas sobre el contenido de `jsonb_array_elements` y estima **16.5 millones de grupos** frente a los 96 reales. Con esa estimación descarta `HashAggregate`, que no cabría en 4 MB, y elige ordenar por país las 500 000 filas *completas* (`width=1460` bytes, el documento entero) antes de desanidar. Los 3 procesos vuelcan en conjunto ≈705 MiB por ejecución, lo que coincide con los 30–81 GiB de archivos temporales medidos por ventana (tabla 10).

Con la variante `relaxed` (64 MB), el `Incremental Sort` pasa a memoria (`quicksort`, pico 42 MB), pero el `Sort` paralelo sigue en disco (≈231–245 MB por proceso, mayor que 64 MB): el volcado se reduce, no desaparece.

5.2 Q1 en MongoDB: agregación en flujo

```
$cursor  COLLSCAN  totalDocsExamined: 500000  (proyección temprana: customer.geo.country, items)
$unwind   nReturned: 1499046
$match    nReturned: 644159
$group    nReturned: 96    peakTrackedMemBytes: 64256   usedDisk: false   spills: 0
executionTimeMillis: 3486   (perfil A: 3229)
```

MongoDB proyecta sólo los dos campos necesarios, desanida en flujo y acumula directamente en una tabla hash de 96 entradas (≈64 KB). No necesita ordenar nada, por eso `allowDiskUse` es irrelevante para Q1 y el plan es idéntico en ambas variantes.

5.3 Q2 en ambos motores: Top-N acotado y patrón de acceso

```
-- PostgreSQL
Limit -> Gather Merge (Workers Launched: 2)
  -> Sort  Sort Method: top-N heapsort  Memory: 276kB   (workers: 273kB, 272kB)
     -> Parallel Seq Scan on orders (rows estimadas 474; reales 60436 x 3)
Execution Time: 2561 ms   (perfil A: 2201 ms)
-- MongoDB
PROJECTION <- SORT (limitAmount: 1000, peakTrackedMemBytes: 1347009, usedDisk: false)
  <- FETCH (226680 docs) <- IXSCAN timestamp_1 (226680 claves)
executionTimeMillis: 9098   (perfil A: 5591)
```

Ambos motores reconocen el `LIMIT 1000` y mantienen sólo los 1 000 mejores candidatos: 276 KB en PostgreSQL y 1.35 MB en MongoDB, muy por debajo de 4 MB y de 100 MB. Por eso Q2 nunca volcó y la variante de memoria no tuvo efecto: **Q2 funciona como control negativo de memoria**, no como estrés de buffer.

La diferencia de rendimiento la explica el patrón de acceso:

- **MongoDB** usa el índice de `timestamp`, pero el filtro selecciona el 45 % de la colección; esto implica 226 680 lecturas aleatorias de documentos (`FETCH`) para evaluar `payment.status`. Cuando la caché de WiredTiger se reduce a 0.5 y 0.25 GB, esas lecturas dejan de estar en memoria y el tiempo pasa de 5.6 s a 9.1 s por ejecución.

- **PostgreSQL** ignora el índice y hace un recorrido secuencial paralelo, que con esa selectividad es más barato y menos sensible a la caché.

6 Interpretación: límites de memoria, volcado a disco y throttling de cgroups

6.1 M1 — ¿Cómo cambian P95/P99 al limitar CPU y memoria?

Tabla 9: M1: P99 (s) por perfil y variación respecto del perfil A, a igual concurrencia (variante `strict`).

Q	Motor	c	P99 A	P99 B	ΔB	Thr. B (s)	P99 C	ΔC	Thr. C (s)
Q1	MongoDB	1	1.95	2.15	+10 %	0	2.60	+33 %	12
Q1	MongoDB	10	8.72	14.80	+70 %	597	31.35	+259 %	894
Q1	MongoDB	50	42.50	63.19	+49 %	595	127.1	+199 %	893
Q1	PostgreSQL	1	7.97	3.82	-52 %	103	8.28	+4 %	219
Q1	PostgreSQL	10	83.19	59.29	-29 %	446	146.8	+76 %	740
Q1	PostgreSQL	50	204.6	—	OOM	424	—	OOM	787
Q2	MongoDB	1	0.71	3.24	+353 %	0	6.53	+813 %	17
Q2	MongoDB	10	2.84	16.98	+497 %	585	55.05	+1836 %	821
Q2	MongoDB	50	11.53	98.31	+753 %	594	334.4	+2802 %	883
Q2	PostgreSQL	1	0.29	0.57	+95 %	316	1.09	+276 %	552
Q2	PostgreSQL	10	3.42	7.66	+124 %	540	21.96	+542 %	643
Q2	PostgreSQL	50	11.55	30.74	+166 %	583	196.7	+1603 %	717

$\Delta = (\mathrm{P99}_X - \mathrm{P99}_A)/\mathrm{P99}_A$. En PostgreSQL Q1 a $c = 1$ el P99 del perfil A está inflado por una repetición con cola larga (rango 3.46–9.32 s) y por un tamaño de muestra menor a 100.

- **La contención de CPU domina la cola.** Para MongoDB en Q1, el P99 a 10 clientes crece 70 % en B y 259 % en C, en paralelo con el *throttling* (597 y 894 s). Con un solo cliente la variación es pequeña (+10 % y +33 %) porque una consulta de MongoDB usa un solo hilo y no agota la cuota.
- **La contención de memoria cambia el tipo de fallo.** En PostgreSQL Q1, a $c = 50$ el problema deja de ser la latencia y pasa a ser la estabilidad: OOM en B y en C.
- **En Q2 la degradación de MongoDB es la mayor de todo el experimento** (+753 % en B y +2 802 % en C a $c = 50$), porque al reducir el perfil se reduce también la caché de WiredTiger y el plan depende de lecturas aleatorias. PostgreSQL se degrada menos (+166 % y +1 603 %).

6.2 M2 — ¿Qué motor gestiona mejor los buffers antes del volcado o del OOM?

Tabla 10: M2: estado de las 3 repeticiones por celda y volumen de archivos temporales de PostgreSQL en la ventana (mediana).

Q	Motor	Perfil	Variante	$c = 1$	$c = 10$	$c = 50$
Q1	MongoDB	A	strict	ok:3	ok:3	ok:3
Q1	MongoDB	B	strict	ok:3	ok:3	ok:3
Q1	MongoDB	C	strict	ok:3	ok:3	ok:3
Q1	MongoDB	C	relaxed	ok:3	ok:3	ok:3
Q1	PostgreSQL	A	strict	spill:3 (81 GiB)	spill:3 (55 GiB)	spill:3 (43 GiB)
Q1	PostgreSQL	B	strict	spill:3 (73 GiB)	spill:3 (55 GiB)	oom:3
Q1	PostgreSQL	C	strict	spill:3 (35 GiB)	spill:3 (30 GiB)	oom:3
Q1	PostgreSQL	C	relaxed	spill:3 (37 GiB)	oom:3	oom:3
Q2	MongoDB	A	strict	ok:3	ok:3	ok:3
Q2	MongoDB	A	relaxed	ok:3	ok:3	ok:3
Q2	MongoDB	B	strict	ok:3	ok:3	ok:3
Q2	MongoDB	B	relaxed	ok:3	ok:3	ok:3
Q2	MongoDB	C	strict	ok:3	ok:3	ok:3
Q2	MongoDB	C	relaxed	ok:3	ok:3	ok:3
Q2	PostgreSQL	A	strict	ok:3	ok:3	ok:3
Q2	PostgreSQL	A	relaxed	ok:3	ok:3	ok:3
Q2	PostgreSQL	B	strict	ok:3	ok:3	ok:3
Q2	PostgreSQL	B	relaxed	ok:3	ok:3	ok:3
Q2	PostgreSQL	C	strict	ok:3	ok:3	ok:3
Q2	PostgreSQL	C	relaxed	ok:3	ok:3	ok:3

Los GiB son el incremento de `pg_stat_database.temp_bytes` en los 300 s de medición. En las celdas `oom` ese contador no es utilizable: la recuperación tras el OOM lo reinicia a cero.

MongoDB gestionó mejor los buffers en esta carga: 0 volcados y 0 OOM en 90 corridas, con una huella de estado de agregación de 64 KB (Q1) y 1.35 MB (Q2). PostgreSQL volcó en el 100 % de las corridas de Q1 que terminaron y alcanzó el OOM en 12 corridas. Las causas son dos:

1. **Una estimación de cardinalidad errónea** sobre la salida de `jsonb_array_elements` lleva a un plan que ordena el documento completo en lugar de agregar en hash. Es una limitación de JSONB (sin estadísticas sobre el contenido), no del almacenamiento en sí.
2. **`work_mem` se aplica por operación y por proceso.** Con 50 sesiones, cada una con líder y 2 *workers* y varios nodos de ordenamiento, la memoria privada se multiplica hasta superar el límite del cgroup.

Hipótesis dual sobre `relaxed`: confirmada. En el perfil C (tabla 8 y figura 4), `work_mem = 64MB` aumentó el throughput de PostgreSQL con 1 cliente de 0.143 a 0.177 QPS (+24 %; menos pases de mezcla). Pero con 10 clientes convirtió el `spill` en `oom` en las 3 repeticiones (60 eventos `oom_kill`): a mayor límite por operación, más memoria privada concurrente y menos margen bajo un techo fijo sin swap. Con 50 clientes ambas variantes terminan en OOM. En MongoDB y en Q2 la variante no cambió el plan.

6.3 M3 — Huella de CPU y memoria por cada 1 000 documentos

Tabla 11: M3 (variante `strict`, mediana de 3 repeticiones). Mo = MongoDB, PG = PostgreSQL.

			CPU_{1k} (ms CPU)		Mem_{peak} (MB)		Mem_{1k} (MB·s)	
Q	c	Perfil	Mo	PG	Mo	PG	Mo	PG
Q1	1	A	3.12	9.90	681	930	2.10	3.74
		B	3.34	9.85	663	651	2.08	3.44
		C	3.86	13.44	427	330	1.59	3.34
	10	A	5.77	17.84	738	1 394	1.08	13.52
		B	4.72	20.09	715	1 130	1.64	11.29
		C	4.95	21.26	453	856	2.19	18.14
	50	A	5.98	22.88	922	2 640	1.38	27.57
		B	4.62	—	805	1 991	1.82	—
		C	4.00	—	607	997	2.33	—
Q2	1	A	1.07	1.44	687	864	0.73	0.43
		B	5.38	1.48	624	582	3.12	0.43
		C	11.29	1.69	387	313	4.15	0.53
	10	A	1.62	1.61	713	1 009	0.29	0.41
		B	5.20	1.80	623	727	1.61	0.65
		C	5.98	2.58	387	458	2.30	1.17
	50	A	1.68	1.46	803	1 434	0.34	0.52
		B	6.11	1.43	721	1 148	2.19	0.82
		C	11.98	3.67	479	871	5.75	3.18

CPU_{1k} y Mem_{1k} se normalizan por consultas completadas × 500 000 documentos; no se definen cuando no se completó ninguna consulta (celdas `oom`). Mem_{peak} es un nivel y no se normaliza: incluye `shared_buffers` (PostgreSQL) y la caché de WiredTiger (MongoDB).

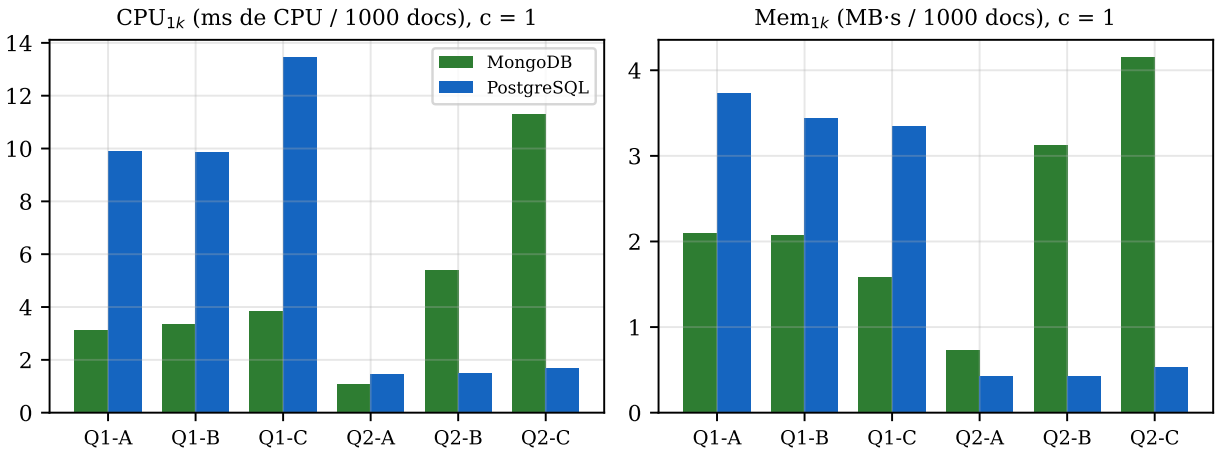


Figura 5: M3 con un cliente: CPU y memoria integrada por cada 1 000 documentos.

- **Q1:** PostgreSQL consume entre 2.9 y 3.5 veces más CPU por cada 1 000 documentos que MongoDB a $c = 1$ (9.9–13.4 frente a 3.1–3.9 ms). La diferencia crece con la concurrencia (17.8–22.9 ms a $c \geq 10$) porque los procesos compiten por E/S de volcado. Es decir, el costo de desanidar y ordenar documentos JSONB completos es mayor que el de recorrer BSON con proyección temprana.
- **Q2:** PostgreSQL es más eficiente y estable (1.4–1.7 ms en todos los perfiles a $c = 1$). MongoDB es comparable en A (1.1 ms) pero sube a 5.4 y 11.3 ms en B y C: las lecturas aleatorias que fallan en caché se pagan en CPU (descompresión de páginas) y en tiempo.
- **Memoria:** el pico de PostgreSQL incluye `shared_buffers` y crece con la concurrencia (hasta 2.6 GB en Q1, A, $c = 50$); el de MongoDB está acotado por la caché de WiredTiger (387–

922 MB). La memoria integrada por documento (Mem_{1k}) favorece a MongoDB en Q1 y a PostgreSQL en Q2, igual que la CPU.

6.4 Throttling de cgroups y paralelismo intra-consulta

El *throttling* reportado puede superar los 300 s porque el kernel lo acumula por CPU del `cpuset`; su techo es $(4 - \text{cuota}) \times 300\,\text{s}$. Dos lecturas se desprenden de los datos:

- Con una cuota de 1 o 2 CPUs y 4 CPUs disponibles en el `cpuset`, el paralelismo intra-consulta de PostgreSQL (`max_parallel_workers_per_gather = 2`) no aporta capacidad: los 3 procesos agotan la cuota antes y pasan el resto del periodo bloqueados. En Q2 con 1 cliente, PostgreSQL acumula 552 s de *throttling* en C, mientras MongoDB, de un solo hilo por consulta, acumula 17 s.
- Con suficientes clientes, ambos motores llegan al techo: la cuota CFS, no el motor, es el cuello de botella. A partir de ese punto, más concurrencia sólo alarga la cola (figura 1).

6.5 Respuestas a las preguntas del revisor

1. **¿El baseline es comparable (mismo dataset, mismas consultas)?** Sí. Ambos motores cargaron el mismo NDJSON (SHA-256 verificado) y devolvieron los mismos conteos y la misma suma de `final_total`; las consultas son las del playbook sin modificaciones; las imágenes están fijadas por digest; el perfil A usa la misma configuración de caché por fabricante que B y C.
2. **¿Las latencias son P50/P95/P99 o sólo promedios?** Son percentiles calculados sobre cada consulta individual de la ventana. Se reporta n por celda y se marcan con † las 19 de 60 celdas con alguna repetición de menos de 100 consultas.
3. **¿Hubo throttling CFS? ¿Cuántos segundos?** Sí, en los perfiles B y C: hasta 597 s y 894 s por ventana, coherentes con el techo teórico de 600 y 900 s (tablas 6, 7 y 9; figura 2). En el perfil A fue prácticamente nulo ($\leq$3 s).
4. **¿Spill a disco u OOM? ¿En qué perfil y concurrencia?** Volcado: PostgreSQL Q1 en todas las celdas que terminaron (A, B y C; $c = 1$ y 10, y $c = 50$ en A). OOM: PostgreSQL Q1 con $c = 50$ en B y C (`strict`) y con $c = 10$ y 50 en C (`relaxed`). MongoDB y Q2: ninguno (tabla 10).
5. **¿Cuántas repeticiones y cómo se resume?** Tres por celda; se reporta la mediana y el rango [mín, máx]. No se promedian percentiles.
6. **¿El plan de ejecución respalda el cuello de botella?** Sí: `external merge` con ≈705 MiB por ejecución y una estimación de 16.5 M de grupos en PostgreSQL Q1; agregación hash de 64 KB en MongoDB Q1; `top-N heapsort` de 276 KB frente a IXSCAN + 226 680 `FETCH` en Q2 (sección 5).

6.6 Amenazas a la validez y limitaciones

1. **Entorno virtualizado en lugar de Linux nativo.** Los contenedores corrieron sobre el kernel Linux de la VM de WSL2 (Docker Desktop), no sobre un anfitrión Linux. Los mecanismos medidos (CFS, `memory.max`, OOM-killer, `cpu.stat`, `memory.stat`) son los del kernel Linux, pero:
   - las CPUs del `cpuset` son CPUs virtuales que Hyper-V reparte entre núcleos P y E del procesador híbrido, por lo que el rendimiento por CPU no es constante;
   - la E/S de los volcados de PostgreSQL atraviesa el disco virtual de la VM, lo que probablemente penaliza más a PostgreSQL que un disco nativo;
   - el control de ruido (CPUs ociosas) sólo observa la VM, no la carga de Windows.

Ambos motores se midieron en el mismo entorno, con orden aleatorio y los mismos filtros de calidad, por lo que las *comparaciones* entre motores y perfiles son válidas; los valores absolutos no deben extrapolarse a un servidor Linux.

2. **Runner fuera de la VM.** Los clientes corrieron en Windows y llegaron a los motores por el reenvío de puertos de Docker Desktop. Esto añade latencia de red local del orden de milisegundos (despreciable frente a latencias de segundos) y compite por los mismos núcleos físicos que la VM.
3. **Degradación periódica de Docker Desktop.** Durante la campaña, Docker Desktop/WSL2 se degradó tras ≈1–1.5 h de uso continuo (lecturas lentas de cgroups, comandos de Docker lentos). Esto produjo 31 intentos descartados y 3 detenciones, y se mitigó con 18 pausas para reiniciar Docker Desktop y WSL entre corridas (anexo A). Ninguna corrida degradada llegó al CSV: todas fallaron el control de muestras y se repitieron.
4. **Censura por la ventana de 300 s.** En las celdas con consultas de decenas o cientos de segundos, las consultas que no terminan dentro de la ventana no se observan, por lo que el P99 está truncado (19 celdas con $n < 100$, marcadas con †). Las celdas `oom` no tienen latencias.
5. **Variabilidad entre corridas.** En celdas donde la variante no cambia el plan, las diferencias llegaron a ±20 % (tabla 8); diferencias menores a ese margen no deben interpretarse como efectos.
6. **Cambios operativos durante la campaña.** El plan se amplió de 144 a 180 corridas (Q2 `relaxed` en A y B) y el tope de incidencias del orquestador se elevó de 4 a 12. Ninguno modificó consultas, perfiles, telemetría ni código de medición: un manifiesto SHA-256 de 21 archivos protegidos se verificó intacto antes de cada reanudación, y las 144 corridas originales conservaron su orden.
7. **Índice de `timestamp` en PostgreSQL** habilitado modificando `pg_catalog`; no es reproducible en servicios administrados (aunque el planificador no lo usó en Q2).

7 Recomendaciones técnicas para despliegues documentales en la nube

7.1 Respaldadas directamente por los resultados

1. **Elegir el motor según el patrón de consulta, no por el modelo de datos.** Para agregaciones que desanidan arreglos y agrupan (tipo Q1), MongoDB obtuvo entre 1.9 y 7.4 veces más throughput, sin volcado y sin OOM. Para filtros amplios con Top-K (tipo Q2), PostgreSQL fue más rápido y menos sensible a la reducción de memoria.
2. **Dimensionar `work_mem` contra el límite del contenedor, no contra una consulta aislada.** Con `memory.swap.max = 0`, una cota práctica es

$$\texttt{work_mem} \lesssim \frac{\text{límite} - \texttt{shared_buffers}}{\text{sesiones} \times (1 + \text{workers}) \times \text{nodos de ordenamiento}}.$$

   Subirlo de 4 a 64 MB convirtió volcados en OOM con 10 sesiones en 1 GB.
3. **Limitar el paralelismo intra-consulta cuando la cuota CFS es pequeña.** Con 1–2 vCPU, `max_parallel_workers_per_gather` en 0 o 1 evita que 3 procesos por consulta se estrangulen mutuamente (552 s de *throttling* con un solo cliente en Q2, perfil C).
4. **Limitar la concurrencia en PostgreSQL con un pool** (por ejemplo, PgBouncer) en contenedores pequeños: el OOM apareció con 50 sesiones en 2 GB, no con 10.
5. **En MongoDB, dimensionar la caché de WiredTiger según el conjunto de trabajo de las consultas con `FETCH` aleatorio.** Reducirla de 1.5 a 0.25 GB multiplicó por 9 la latencia de Q2 a 1 cliente; fijar siempre `--wiredTigerCacheSizeGB` según el límite del contenedor.

6. **Medir con percentiles y *throttling*, no sólo con CPU promedio.** El *throttling* explicó la cola de latencia aun cuando el uso de CPU parecía igual a la cuota.

7.2 Buenas prácticas generales (no medidas directamente)

- En PostgreSQL con JSONB, materializar los campos que se agrupan o filtran con frecuencia (columnas generadas `STORED` o tablas normalizadas) para que el planificador tenga estadísticas y elija `HashAggregate`.
- En servicios administrados (RDS, Cloud SQL, Azure) no se puede modificar `pg_catalog`: para indexar fechas en JSONB, usar una función `IMMUTABLE` propia o extraer el valor a una columna `timestamptz`.
- Usar `allowDiskUse: true` en MongoDB sólo con concurrencia acotada en etapas `$group`/`$sort` de alta cardinalidad.
- Validar en el entorno de producción (Linux nativo o el servicio administrado) los valores absolutos de este estudio, obtenidos en una VM de WSL2.

A Registro de incidencias de la campaña

La campaña empezó el 31 de octubre de 2026 a las 02:02 y terminó el 1 de noviembre a las 14:44 (36.7 h de reloj). La tabla 12 resume el journal del orquestador (`state/journal.jsonl`).

Tabla 12: Eventos operativos registrados en el journal.

Corridas iniciadas / registradas	211 / 180
Intentos descartados y repetidos	31
por `telemetry_gap` (menos de 240 muestras)	24
por `runner_exception` (clientes sin sincronizar en 60 s)	3
por `compose_failed` / `engine_start_failed` / `drop_caches_failed`	2 / 1 / 1
Incidencias abiertas por el orquestador	9
Detenciones (`halted`)	3
Pausas para reinicio preventivo de Docker Desktop y WSL	18

Los intentos descartados no se escribieron en el CSV. Tras cada detención o reinicio se verificó con `doctor` que el manifiesto SHA-256 de los archivos protegidos y el dataset no hubieran cambiado antes de reanudar.

B Reproducibilidad

```
python synthetic_data_generator.py --n 500000 --seed 42 --out-dir data   # SHA-256 9cd202a6...
docker compose pull
./run_benchmarks.sh doctor          # verificaciones previas
./run_benchmarks.sh load-data       # carga y verificación de paridad
./run_benchmarks.sh start           # 180 corridas, plan con semilla 20261030
./run_benchmarks.sh summarize       # medianas y rangos por celda
```

Archivos de evidencia:

- `results/mediciones.csv` (108 filas) y `results/anexo_variantes.csv` (72 filas);
- `results/raw/<run_id>/` con `meta.json`, `series.csv` (1 Hz) y `latencies.jsonl` (cada consulta);
- `results/explain/` (24 planes);
- `state/journal.jsonl` y `state/plan.json`.

C Esquema del CSV de mediciones

```
timestamp_epoch,engine,profile,query_id,concurrency,run_number,throughput_qps,
latency_p50_ms,latency_p95_ms,latency_p99_ms,cpu_throttled_seconds,max_rss_mb,disk_spill,status
```

Esquema idéntico al de la sección 8.2 del playbook. `query_id` toma los valores `Q1`, `Q2`, `Q1_relaxed` y `Q2_relaxed`; `status` toma `ok`, `spill`, `oom` o `error`.